

22/25

**The 3 I missed taught me
more than the 22 I got right.**

A code-comprehension test I ran
on myself. Here's what it caught.

Interviews are reportedly grading code reading - not code writing.

Reading and debugging an existing codebase, with an AI assistant available — instead of writing from a blank file.

*Per a few interview-prep sources I've read.
Not a round I've sat myself.*

25 questions.

Read the code.

Reason about it.

Predict the output.

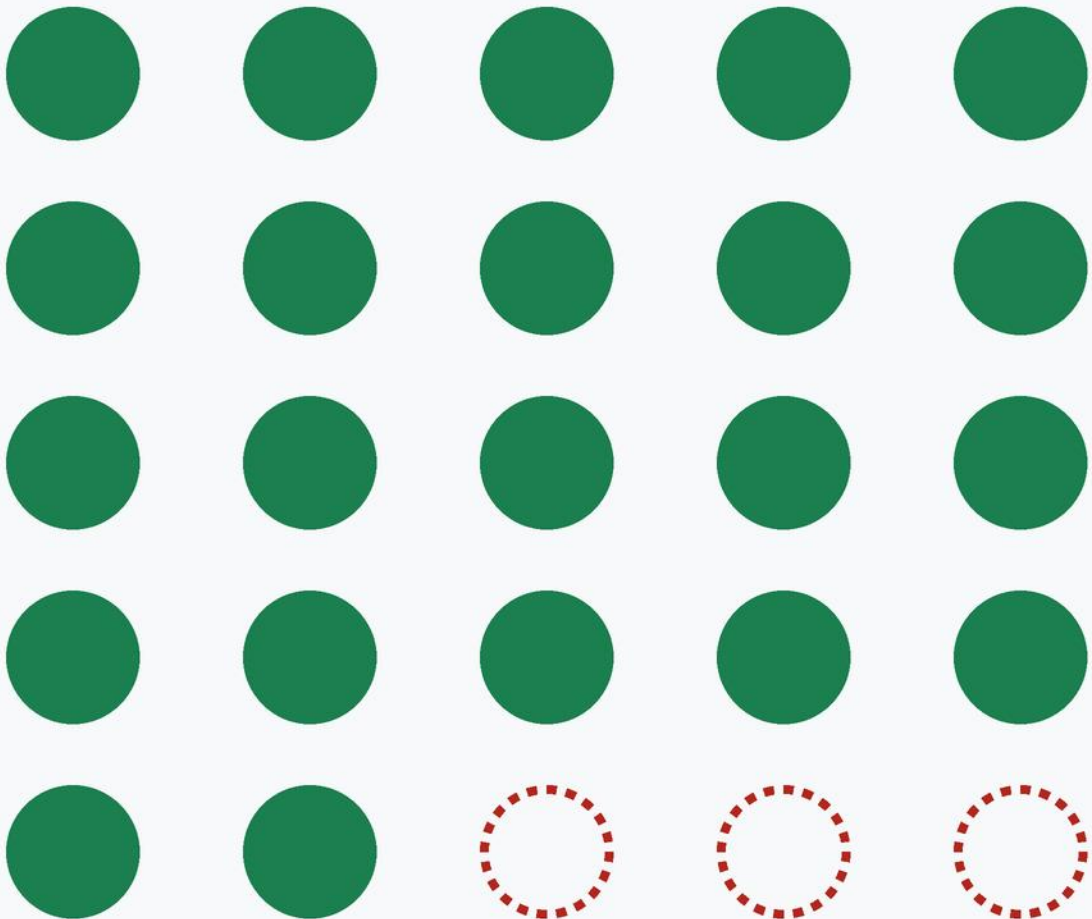
No writing from scratch.

No LLM open.

Python semantics, collections,
concurrency, SQL.

THE SCORE

22 right. 3 wrong.



 correct

 missed

All three were Python.

```
[x for x in xs if f(x)]
```

I misjudged what actually got filtered.

```
{[1, 2]: "value"}
```

Why a mutable type can't be a dict key.

```
sorted(xs, key=f)
```

Whether ties keep their original order.

I knew all three concepts.

Concept gap

vs.

fluency gap.

**I knew every one of those
three concepts cold.**

I was just slower and less
certain reading the actual
code under pressure.

**“I understand hashability”
and “I read that line right
in 4 seconds under pressure”
are two different skills.**

**Right now I only have real
evidence for the first one.**

**Have you tried a comprehension
test on your own code?
What did it catch?**